

IoT-Based Weather Monitoring and Alert System Using ESP32, DHT22 and ThingSpeak

Student name: Rudrapriya Rajesh Choudhari

Institution: IIT Bhubaneswar

Department/Program: Dual Degree in Mechanical Engineering

(Specialization: System Design)

Project type: IoT Weather Monitoring System

Date: August 2026

Table of Contents

Table of Contents	2
1. Introduction.....	11
2. Problem Statement	11
3. Objectives	11
4. System Overview	11
5. Hardware Components	12
5.1 ESP32	12
5.2 DHT22	12
6. Software and Platforms.....	12
6.1 Wokwi.....	12
6.2 Arduino Framework.....	13
6.3 ThingSpeak	13
7. System Architecture	13
8. Circuit Design and Connections.....	13
9. Software Implementation	14
9.1 Sensor Initialization	14
9.2 Wi-Fi Connection	14
9.3 Sensor Data Acquisition.....	14
9.4 HTTP Data Transmission.....	15
9.5 ThingSpeak Data Fields.....	15
10. Cloud Monitoring and Visualization	15
11. Threshold-Based Email Alert System.....	16
12. Testing Methodology.....	17
13. Results and Observations	17
14. Challenges and Troubleshooting	20
15. Limitations	20
16. Future Improvements.....	20
17. Conclusion	21
18. References	21

I need you to make a FINAL POLISH pass on the existing Word report:

IoT_Weather_Monitoring_System_Report.docx

IMPORTANT:

- DO NOT recreate the report from scratch.
- DO NOT change the overall structure unnecessarily.
- DO NOT remove technical content that is already present.
- DO NOT modify the actual project implementation description.
- DO NOT invent new results, experiments, hardware, or functionality.
- Preserve the existing images unless specifically instructed below.
- Create a NEW edited copy rather than overwriting the original.
- Keep the final document fully editable in Microsoft Word.

Create the edited file as:

IoT_Weather_Monitoring_System_Report_FINAL.docx

=====

FIX 1 — TABLE OF CONTENTS

=====

The current Table of Contents has incorrect page numbers.

Update/rebuild the Table of Contents so that ALL page numbers correspond to the actual final page locations.

IMPORTANT:

- Do not simply type approximate page numbers.
- Use a proper Word automatic Table of Contents if possible.
- Update the TOC after all other modifications are complete.
- Make sure headings remain properly associated with the TOC.
- Verify that the displayed page numbers match the actual document pages.

=====

FIX 2 — REMOVE REPETITIVE THINGSPEAK SCREENSHOT

=====

The report currently contains two somewhat repetitive ThingSpeak figures:

- Figure 3: ThingSpeak Channel overview
- Figure 4: ThingSpeak Interface and Dashboard

Keep the stronger/more useful screenshot.

Preferably:

- Keep the screenshot showing the actual ThingSpeak channel/dashboard.
- Remove the redundant/repetitive screenshot if it does not add meaningful information.

IMPORTANT:

There is a screenshot in the media folder showing the final dashboard with:

- Temperature gauge
- Humidity gauge
- Temperature trend
- Humidity trend

If that image is available, use it as the main ThingSpeak dashboard figure because it provides stronger evidence of the completed system.

Update figure numbering and references in the text automatically after removing/replacing the image.

=====

FIX 3 — REDUCE GENERIC / AI-LIKE WORDING

=====

Make the language slightly more natural and technically professional.

Do NOT rewrite the whole report.

Only replace unnecessarily exaggerated or generic phrases.

For example, avoid phrases such as:

"robust proof-of-concept"

"flawlessly"

"excellent foundation"

"robust real-time data visualization"

Prefer concise engineering language such as:

"The system was successfully validated in the Wokwi simulation environment."

"The ThingSpeak dashboard successfully displayed the uploaded sensor data."

"The implementation demonstrates the complete sensor-to-cloud data pipeline."

Keep the report academic and professional, but make it sound like a student engineering project report rather than marketing material.

Do not over-edit the writing.

=====

FIX 4 — TECHNICAL STATEMENT ABOUT THE DHT22

=====

Find the statement:

"The simulation handles the required pull-up resistor internally on the data line."

Remove that statement unless it can be directly verified from the actual Wokwi implementation.

Replace it with a simple factual description:

"The DHT22 data pin was connected to GPIO 15, with VCC connected to 3.3 V and GND connected to the ESP32 ground."

Do not introduce additional electrical claims that were not actually tested.

=====

IMPORTANT TECHNICAL CONSISTENCY CHECK

=====

Before saving the final document, verify that the report consistently states:

DHT22 DATA → ESP32 GPIO 15

Field 1 → Temperature

Field 2 → Humidity

Temperature alert threshold → greater than 35 ° C

Humidity alert threshold → greater than 80 %

Data upload interval → 20 seconds

Simulation platform → Wokwi

Cloud platform → ThingSpeak

The project was NOT physically built.

Do not change any of these values.

=====

FIGURE QUALITY

=====

Check all figures after editing.

Make sure:

- Images are not stretched.
- Images are not unnecessarily tiny.
- Captions remain directly below their figures.
- Figure numbering is sequential.
- Figure references in the text remain correct.
- Screenshots are clear enough to read.
- No screenshot is duplicated unnecessarily.

Do not add decorative images.

=====

FINAL FORMATTING CHECK

=====

Keep the current professional engineering-report style.

Check:

- Heading hierarchy
- Font consistency
- Paragraph spacing
- Margins
- Page numbers
- Header/footer
- Figure captions
- Table captions
- Table of Contents
- Section numbering
- Page breaks

Avoid excessive formatting changes.

=====

FINAL VALIDATION

=====

After editing:

1. Recalculate/update the Table of Contents.
2. Verify all page numbers.
3. Verify figure numbering.
4. Verify all figure references.
5. Check that no API key or secret credential appears anywhere.
6. Check that the report does not claim physical hardware testing.
7. Check that GPIO 15 is used consistently.
8. Check that the alert thresholds are correct.
9. Check that the final dashboard screenshot is used where appropriate.
10. Check the document visually page-by-page.
11. Make sure there are no blank pages or awkward page breaks.
12. Save as a NEW file:

IoT_Weather_Monitoring_System_Report_FINAL.docx

DO NOT modify or delete the original report.

1. Introduction

With the rapid growth of the Internet of Things (IoT), environmental monitoring systems have become highly accessible and efficient. This project presents a simulated IoT-based weather monitoring and alert system that measures temperature and humidity using a DHT22 sensor interfaced with an ESP32 microcontroller. The system sends real-time sensor data over a Wi-Fi connection to the ThingSpeak cloud platform for storage, analysis, and visualization. Furthermore, a threshold-based email alert mechanism is configured on the cloud side to notify users when environmental parameters exceed safe limits. The entire project was developed and validated in the Wokwi simulation environment, demonstrating the complete sensor-to-cloud data pipeline.

2. Problem Statement

Monitoring environmental parameters like temperature and humidity is critical in various domains, including agriculture, warehousing, home automation, and industrial processes. Traditional monitoring systems often lack real-time data visibility and remote alert capabilities, requiring manual supervision. The challenge is to design a cost-effective, scalable, and connected monitoring solution that not only tracks environmental data in real-time but also triggers automated alerts when conditions fall outside of normal operating bounds, thereby enabling proactive intervention.

3. Objectives

The primary objectives of this project are to:

- 1. Read precise temperature and humidity data using a DHT22 sensor.
- 2. Employ an ESP32 microcontroller as the central processing and Wi-Fi communication node.
- 3. Establish a stable Wi-Fi connection to transmit the collected data.
- 4. Transmit the sensor readings to the ThingSpeak cloud platform via HTTP requests.
- 5. Display the received data in the form of live and time-series visualizations on a cloud dashboard.
- 6. Implement and test an automated email alert mechanism that triggers when predefined thresholds are crossed.

4. System Overview

The system consists of an ESP32 microcontroller and a DHT22 sensor, simulated entirely within the Wokwi environment. The ESP32 collects readings from the sensor at regular intervals and transmits them over a virtual Wi-Fi network to the ThingSpeak platform. ThingSpeak processes the incoming HTTP payloads, stores the data, and displays it on a graphical dashboard. In the

background, ThingSpeak's React/Alert mechanism continuously evaluates the data against predefined thresholds, automatically dispatching an email alert if a threshold violation occurs.

5. Hardware Components

5.1 ESP32

The ESP32 is a low-cost, low-power system on a chip (SoC) featuring integrated Wi-Fi and dual-mode Bluetooth capabilities. In this project, it acts as the primary computational unit, reading digital data from the DHT22 and managing the network connection to post data to the cloud.

5.2 DHT22

The DHT22 is a basic, low-cost digital temperature and humidity sensor. It utilizes a capacitive humidity sensor and a thermistor to measure the surrounding air and outputs a digital signal on its data pin. It was selected for its reliability and ease of interfacing.

6. Software and Platforms

6.1 Wokwi

Wokwi is an online electronics simulator that supports ESP32, Arduino, and other microcontrollers. It provides a virtual environment for wiring components, compiling code, and simulating real-time execution. It was used to simulate both the hardware circuit and the Wi-Fi connectivity required for this project.

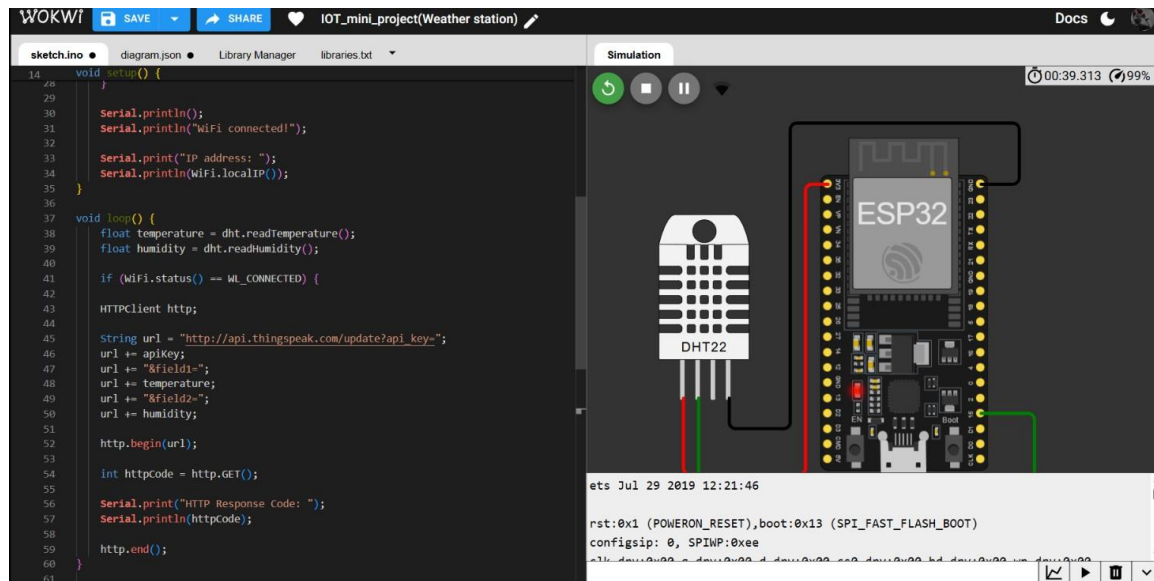


Figure 1. Wokwi Simulation Environment Interface

6.2 Arduino Framework

The Arduino framework (C++ based) was utilized to program the ESP32. Libraries such as 'WiFi.h' for networking, 'HTTPClient.h' for API communication, and 'DHT.h' for sensor interfacing were heavily utilized to simplify the software implementation.

6.3 ThingSpeak

ThingSpeak is an IoT analytics platform service that allows aggregation, visualization, and analysis of live data streams in the cloud. In this project, a ThingSpeak channel was configured to receive and graph the data, and its event-handling capabilities were leveraged to trigger alerts.

7. System Architecture

The flow of data from the sensor to the user's email inbox follows a structured pipeline. The sensor collects raw environmental data, which is processed by the ESP32 and formatted into HTTP packets. These packets are sent to the ThingSpeak Cloud via the Internet. The cloud handles the visualization and threshold evaluation.

DHT22 → ESP32 → Wi-Fi → Internet → ThingSpeak Cloud → Visualization

For alerts:

ThingSpeak → Threshold Checking → Email Alert Mechanism → Configured Recipient

8. Circuit Design and Connections

The circuit was wired within the Wokwi simulator. The DHT22 sensor was connected to the ESP32 with the following pinout configuration:

- - VCC → ESP32 3.3V
- - DATA/SDA → ESP32 GPIO 15
- - NC → Not connected
- - GND → ESP32 GND

The DHT22 data pin was connected to GPIO 15, with VCC connected to 3.3 V and GND connected to the ESP32 ground.

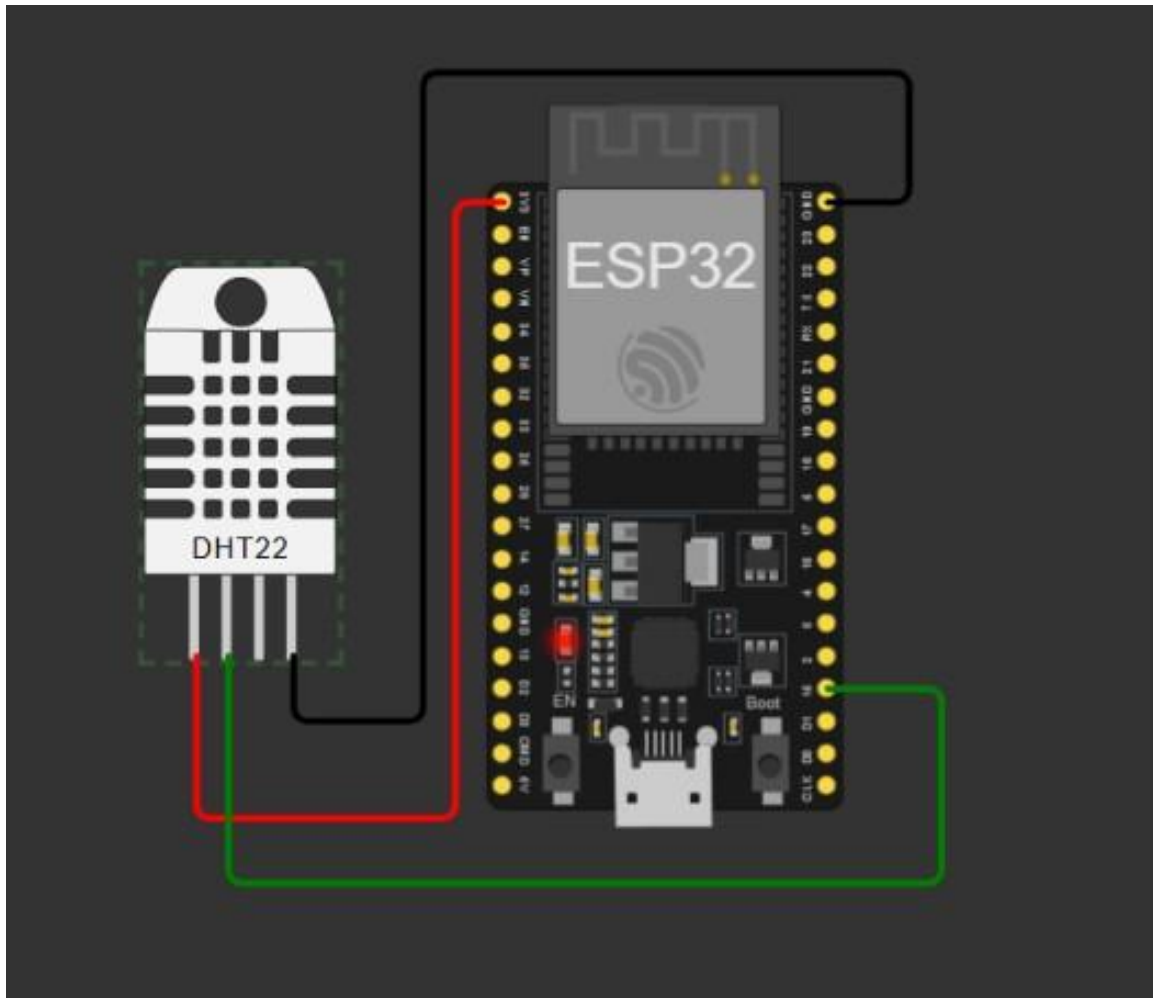


Figure 2. Wokwi simulation circuit showing ESP32 and DHT22

9. Software Implementation

9.1 Sensor Initialization

The DHT22 sensor is initialized by configuring the designated pin (GPIO 15) and specifying the sensor type. Serial communication is also started at a baud rate of 115200 for debugging purposes.

9.2 Wi-Fi Connection

The ESP32 connects to the virtual Wokwi-GUEST Wi-Fi network. The connection status is continuously polled until a valid IP address is assigned.

9.3 Sensor Data Acquisition

In the main loop, temperature and humidity values are read using the DHT library's built-in functions. The readings are checked for valid float values to ensure no 'NaN' results are transmitted.

9.4 HTTP Data Transmission

The system constructs a GET request URL containing the ThingSpeak API endpoint, the Write API key, and the sensor parameters. The HTTPClient library handles the transaction and returns the HTTP response code to confirm successful delivery.

The core logic snippet is as follows:

```
#include <WiFi.h>
#include <DHT.h>
#include <HTTPClient.h>

#define DHT_PIN 15
#define DHT_TYPE DHT22

// Code sends data to:
api.thingspeak.com/update?api_key=YOUR_WRITE_API_KEY&field1=temp&field2=hum
```

9.5 ThingSpeak Data Fields

The ThingSpeak channel was configured to parse the URL parameters into specific fields:

- - Field 1: Temperature (°C)
- - Field 2: Humidity (%)

10. Cloud Monitoring and Visualization

The ThingSpeak channel, named 'ESP32 Weather Station', serves as the graphical front end. It features multiple widgets including a temperature gauge, a humidity gauge, and line graphs mapping the trend of both parameters over time. As the ESP32 pushes updates every 20 seconds, the dashboard refreshes automatically.

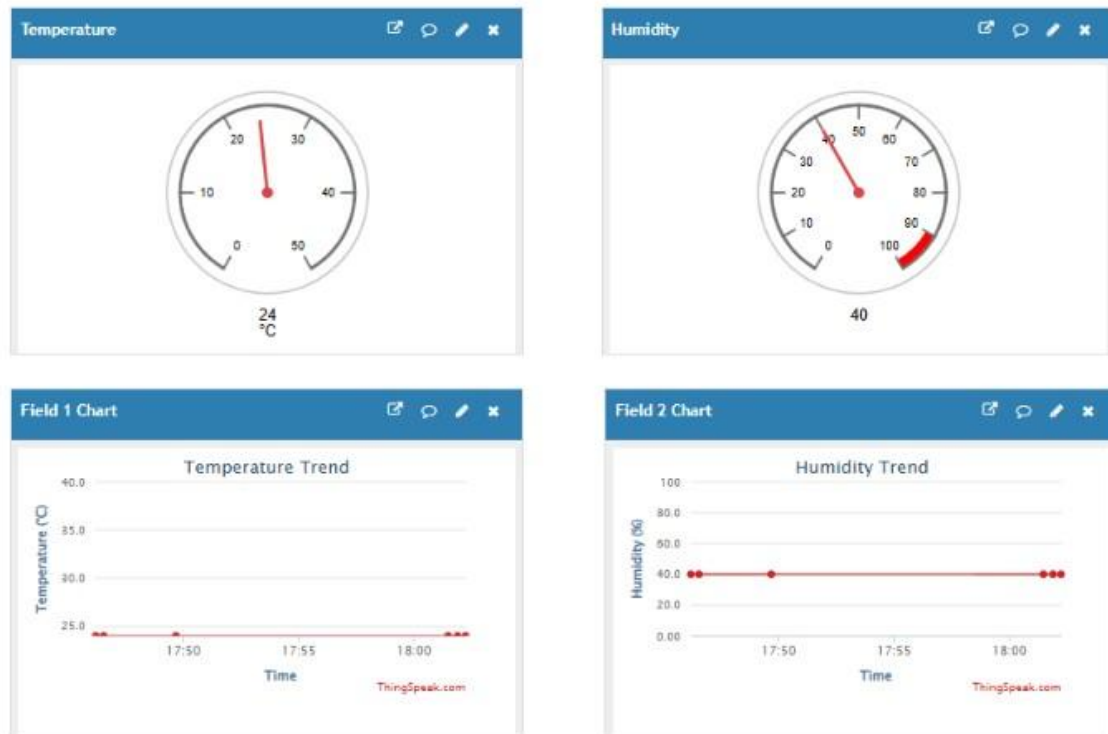


Figure 3. Complete ThingSpeak Dashboard with Gauges and Graphs

11. Threshold-Based Email Alert System

To enable proactive monitoring, a cloud-side alert mechanism was configured. The ESP32 does not directly send emails; instead, it purely functions as a data acquisition node. ThingSpeak handles the conditional logic through its React app / Alert capabilities.

The predefined demonstration thresholds were set to:

- - Temperature > 35 °C
- - Humidity > 80 %

When a new data point arrives, ThingSpeak evaluates it against these thresholds. If a violation is detected, an automated HTTP callback or integrated alert service dispatches an email to the configured recipient.

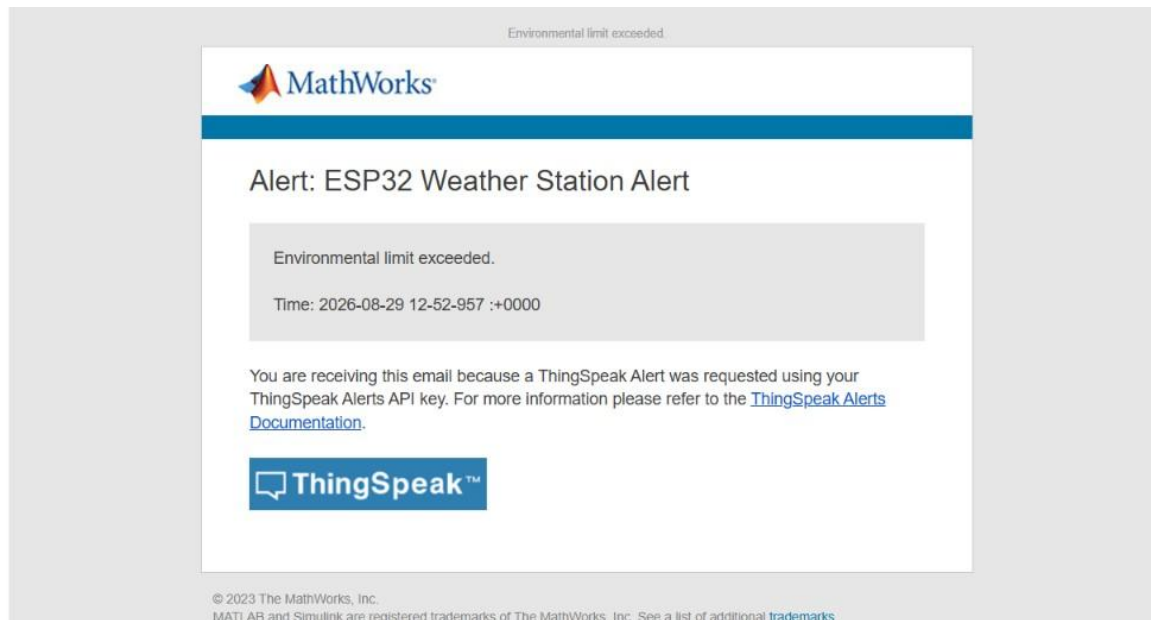


Figure 4. Email alert generated after threshold violation

12. Testing Methodology

Testing was conducted systematically within Wokwi to validate both data transmission and threshold alerts. The interactive nature of the Wokwi DHT22 component allowed for the manual manipulation of sensor values during runtime.

Test Scenario	Action	Expected Outcome
Test 1: Normal Conditions	Temperature set to 25 °C, Humidity 50%	Data successfully read, ThingSpeak updated, no alert triggered.
Test 2: High Temperature	Temperature raised above 35 °C	ThingSpeak receives elevated temperature, threshold triggered, email alert sent.
Test 3: High Humidity	Humidity raised above 80%	ThingSpeak receives elevated humidity, threshold triggered, email alert sent.

13. Results and Observations

The system successfully met all expected outcomes. Serial monitor logs confirmed successful Wi-Fi connection and HTTP 200 response codes for data transmission.

```
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Weather Station Starting...
Connecting to WiFi.....
WiFi connected!
IP address: 10.10.0.2
HTTP Response Code: 200
Temperature: 24.00 °C
Humidity: 40.00 %
HTTP Response Code: 200
Temperature: 24.00 °C
Humidity: 40.00 %
HTTP Response Code: 200
Temperature: 24.00 °C
Humidity: 40.00 %
HTTP Response Code: 200
Temperature: 24.00 °C
Humidity: 40.00 %
```

Figure 5. Serial monitor showing sensor readings and HTTP responses

The cloud dashboard displayed accurate, time-synced visualizations of the manipulated sensor parameters.

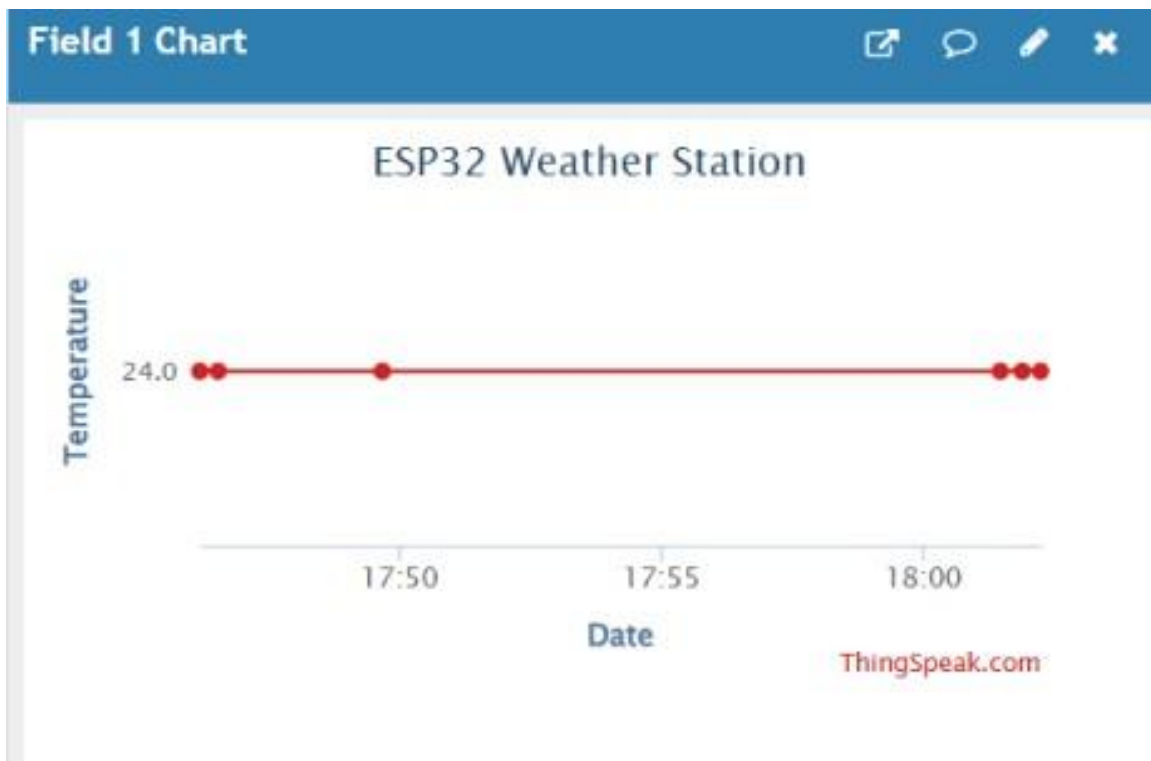


Figure 6. ThingSpeak temperature visualization



Figure 7. ThingSpeak humidity visualization

14. Challenges and Troubleshooting

During development, several engineering challenges were encountered and resolved:

1. Pin Configuration Mismatch: Initial DHT22 readings returned 'NaN' because the software pin definition did not align with the hardware wiring. Correcting the software macro to `#define DHT_PIN 15` resolved the issue.
2. API Initialization: ThingSpeak initially displayed zero entries despite a successful Wi-Fi connection. This was traced to a missing API key integration, which was promptly added to the HTTP payload URL.
3. Dashboard Configuration: The dashboard required manual configuration to properly bind Field 1 to the Temperature widgets and Field 2 to the Humidity widgets. Once configured, the ThingSpeak dashboard successfully displayed the uploaded sensor data.

15. Limitations

The current scope of the project presents the following limitations:

- - Simulation Only: The project was implemented entirely in simulation using Wokwi. No physical ESP32 or DHT22 hardware was deployed.
- - Lack of Physical Constraints: Real-world sensor noise, environmental effects, power consumption, physical Wi-Fi degradation, and hardware reliability over time were not evaluated.
- - Proof of Concept: The system is primarily a proof-of-concept IoT monitoring architecture rather than a production-ready solution.

16. Future Improvements

Although successful in simulation, several future enhancements (FUTURE WORK) could expand the system's capabilities:

- - Hardware Deployment: Deploying the codebase on physical ESP32 hardware for real-world data collection.
- - Additional Sensors: Integrating pressure (BMP280) or air quality (MQ-135) sensors.
- - Power Efficiency: Implementing sleep modes for battery or solar-powered deployment.
- - Protocol Upgrade: Transitioning from HTTP to MQTT for lighter, more efficient telemetry transport.
- - Advanced Alerts: Adding configurable threshold settings via a custom web/mobile dashboard and implementing sensor fault detection algorithms.
- - Ruggedization: Designing an outdoor weatherproof enclosure for the physical hardware.

17. Conclusion

In conclusion, the IoT-based weather monitoring system was successfully implemented and simulated using Wokwi, the Arduino framework, and ThingSpeak. The ESP32 effectively read data from the DHT22 on GPIO 15 and reliably transmitted the payloads via HTTP. The integration with ThingSpeak provided real-time data visualization and successfully triggered cloud-side email alerts when temperature or humidity thresholds were exceeded. The implementation demonstrates the complete sensor-to-cloud data pipeline for remote environmental monitoring.

18. References

- [1] Espressif Systems. ESP32 Technical Reference Manual.
- [2] Aosong Electronics. DHT22 (AM2302) Temperature and Humidity Sensor Datasheet.
- [3] Wokwi Simulator Documentation: <https://docs.wokwi.com>
- [4] ThingSpeak API Documentation, MathWorks: <https://mathworks.com/help/thingspeak/>